# SISTEMAS COMPUTACIONAIS AVANÇADOS (SISTCA)

LICENCIATURA EM ENGENHARIA ELETROTÉCNICA E DE COMPUTADORES

INSTITUTO SUPERIOR DE ENGENHARIA DO PORTO
POLITÉCNICO DO PORTO

**Guião das Aulas Praticas:**

## Linguagem de Programação Rust

**Controlo de Versões**

| Versão Número | Data Submissão | Autores | Atualização de Informações |
|---|---|---|---|
| **V1.0** | Março<br><br>2025 | Carlos Oliveira (1201683)<br>Fábio Oliveira (1210838)<br>Simão Fonseca (1210708)<br>Pedro Pereira (1201671)<br><br>Supervisão: Prof. Paula Viana | Versão original do guião. |
| | | | |
| | | | |
| | | | |
| | | | |

**Índice**


1. Introdução ..... 1

1.1 Contexto Científico/Tecnológico ..... 1

1.2 Motivação para este Tópico/Guião de Laboratório ..... 1

1.3 Objetivos ..... 1

1.4 Estrutura ..... 1

2. Teórico ..... 2

2.1 Principais conceitos/terminologia teóricos ..... 2

2.2 Estado de Arte ..... 3

2.2.1 Contexto e Características Técnicas ..... 3

2.2.2 Comparação com Tecnologias Similares ..... 3

3. Perspetivas da tecnologia ..... 5

3.1 Configuração/Instalação (SW/HW) ..... 5

3.1.1 Windows ..... 5

3.1.2 Linux ..... 9

3.2 Primeiros passos ..... 10

4. Tutorial/Funcionalidade ..... 12

4.1 Variáveis ..... 12

4.2 Tipos de Dados ..... 15

4.3 Funções ..... 19

4.3.1 Funções com parâmetros ..... 20

4.3.2 Funções com Valores de Retorno ..... 20

4.4 Control Flow ..... 21

4.5 Borrowing ..... 26

4.6 Structs, Enums e Lifetimes ..... 28

4.7 Threads ..... 29

5. Exercício(s) ..... 31

5.1 Problema A ..... 31

5.1.1 (re)solução A ..... 31
5.2 Problema B ..... 32
5.2.1 (re)solução B ..... 33
5.3 Problema C ..... 35
5.3.1 Resolução C ..... 35
5.4 Problema D ..... 36
5.4.1 Resolução D ..... 36
6. Exercícios Complementares ..... 37
6.1 Problema A ..... 38
6.1.1 Resolução A ..... 39
6.2 Problema B ..... 39
6.2.1 Resolução B ..... 40
7. Desafio ..... 41
8. Referências ..... 42

## Índice de figuras

Figura 1 - Download do executável ..... 5

Figura 2 - Menu 1 de instalação ..... 6

Figura 3 - Instalação do Visual Studio ..... 6

Figura 4 - Visual Studio instalado ..... 7

Figura 5 - Menu 2 de instalação ..... 7

Figura 6 – Rust instalado ..... 8

Figura 7 - Verificar versão do Rust ..... 8

Figura 8-Execução do comando ..... 9

Figura 9-Opções de instalação (Prosseguir com a instalação padrão) .....10

Figura 10-Rust instalado .....10

Figura 11-Versão do rustc .....10

Figura 12-Versão do cargo .....10

Figura 13-Criação do projeto hello_world .....11

Figura 14-Caminho para o projeto .....11

Figura 15-Execução do projeto .....11

Figura 16-Tentativa de atribuição de um valor a uma variável imutavel .....12

Figura 17-Atribuição de um valor com o tipo diferente ao esperado a uma variável mutável .....14

Figura 18 - Demonstra a ordem sequencial das instruções da função main() .....19

Figura 19 - Resultado da função labeled_measurement() .....20

Figura 20 - Resultado da main() .....21

Figura 21-Condição com um tipo diferente de bool .....22

Figura 22-O tipo variável da condição alternativa é diferente da primeira .....23

Figura 23 - Referência inválida .....27

Figura 24 - Visualização de memória .....27

Figura 25 - *Output* código *threads* .....30

Figura 26 - Exemplo do funcionamento .....41

## Índice de tabelas

Tabela 1 - Comparação do Rust com diferentes tecnologias ..... 5

# 1. Introdução

## 1.1 Contexto Científico/Tecnológico

*Rust* é uma linguagem de programação que apareceu pela primeira vez em 2012 com o primeiro lançamento público do *Rust* 0.1, após seis anos de desenvolvimento. Com o patrocínio da *Mozilla*, o *Rust* 1.0 foi publicado a 15 de maio de 2015 e continuou a crescer, recebendo uma série de atualizações até hoje.

A linguagem orgulha-se de ter uma alta segurança de memória e desempenho semelhante ao *C*, tornando-a perfeita para qualquer tipo de aplicação de baixo nível ou software de nível empresarial. Ela alcança essas conquistas com o seu compilador, pois o *Rust* é uma linguagem compilada "*ahead-of-time*", o que significa que, além de o compilador detetar quase todas as vulnerabilidades, também significa que o executável compilado pode ser partilhado e executado sem sequer ter o *Rust* instalado.

Esses aspetos tornam o *Rust* um grande concorrente do *C* e do *C++* na criação de software crítico, que exige alta estabilidade e desempenho.
O *Rust* tem vários aspetos que o tornam distinto de outras linguagens, que serão abordados mais detalhadamente neste documento, como o "*borrow checker*", que está no coração das regras de segurança de memória impostas pela linguagem.

## 1.2 Motivação para este Tópico/Guião de Laboratório

Como foi mencionado anteriormente, o *Rust* pode ser utilizado em qualquer tipo de aplicação de baixo nível, incluindo sistemas embebidos, servidores web e sistemas operativos, que são tópicos abordados no LETI e são de grande interesse. O aumento da adoção do *Rust* também faz dele uma ferramenta útil para aprender para o futuro, uma vez que é uma das linguagens mais apreciadas e está atualmente a ser utilizada no desenvolvimento do *kernel* do *Linux*, juntamente com o *C*.

## 1.3 Objetivos

Após este guia, o estudante deverá ser capaz de entender os principais usos da linguagem *Rust*. As principais diferenças em relação a outras linguagens com objetivos semelhantes. Como criar, compilar e executar um pacote usando o *Cargo*, o gestor de pacotes oficial do *Rust*. Deverá ser também capaz de aprender os fundamentos da linguagem de programação seguindo um conjunto de tutoriais, desenhados para guiar o estudante pelos conceitos básicos do *Rust*. No final do guia, o estudante deverá ser capaz de resolver um desafio que incorpore os principais conceitos abordados neste guia.

## 1.4 Estrutura

Este guião de laboratório está estruturado da seguinte forma:

**Capítulo 1**: Apresenta os conceitos teóricos e o contexto científico/tecnológico relacionados com a linguagem *Rust*.

**Capítulo 2**: Serão abordados os principais conceitos e terminologias, bem como o estado da arte desta tecnologia e uma comparação com outras linguagens de programação semelhantes.

**Capítulo 3**: Foca-se na instalação e configuração do ambiente de desenvolvimento necessário para começar a programar em *Rust*. Será discutido como instalar as ferramentas necessárias em diferentes sistemas operativos (*Windows* e *Linux*) e os primeiros passos para iniciar a programação.

**Subcapítulo 3**.2: Apresenta um tutorial prático, onde o estudante será guiado passo a passo para entender e utilizar as principais funcionalidades da linguagem *Rust*. Este capítulo tem como objetivo a aquisição de habilidades práticas na criação de programas *Rust* simples.

**Capítulo 4**: Aborda de forma detalhada todas as funcionalidades importantes para trabalhar com *Rust*, tais como variáveis, tipos de dados, funções, condições de teste.

**Capítulo 5**: Contém vários exercícios práticos que devem ser resolvidos pelos estudantes utilizando *Rust*. Os problemas propostos serão acompanhados pelas soluções correspondentes, também terá a possibilidade de completar exercícios como forma de aprendizagem.

**Capítulo 6**: Propõe um desafio final, onde o estudante deverá aplicar os conhecimentos adquiridos ao longo do guião para resolver um problema mais complexo, incorporando todos os conceitos abordados.

## 2. Teórico

### 2.1 Principais conceitos/terminologia teóricos

***Rust*:** Linguagem de programação, criada com foco na segurança de memória, concorrência segura e alto desempenho, sem recurso a *garbage collector* [1].

***RustC*:** Compilador do *Rust* [2].

***Crate*:** Unidade básica de compilação em *Rust*, equivalente a um pacote ou biblioteca. Pode ser um projeto binário (executável) ou uma biblioteca reutilizável [3].

***Cargo*:** Gestor de pacotes e sistema de *build* do *Rust*. Usado para criar projetos, compilar, resolver dependências e executar testes [4].

**Compilação *ahead-of-time* (AOT):** *Rust* compila o código diretamente para código de máquina antes da execução, gerando executáveis rápidos e portáteis. Não requer instalação do *Rust* para correr o programa [5].

***Ownership* (Propriedade):** Sistema que define quem "possui" um valor em determinado momento. Quando uma variável recebe um valor, torna-se sua dona. Conceito central para gestão de memória [6].

***Borrowing* (Empréstimo):** É um mecanismo do *Rust* que permite aceder a um valor sem se tornar dono dele (ou seja, sem transferir a *ownership*). Funciona através de referências (como "apontadores seguros"), que "emprestam" o valor temporariamente [7].

***Borrow Checker*:** Ferramenta do compilador que verifica, em tempo de compilação, o cumprimento das regras de *ownership* e *borrowing*. Evita erros como acesso a memória inválida [8].

***Lifetime* (Tempo de Vida):** Define por quanto tempo uma referência é válida. O compilador usa este conceito para prevenir acesso a memória inválida, sem necessidade de *garbage collector* [9].

***Garbage Collector* (GC):** Mecanismo automático de linguagens como *Java* ou *Go* para libertar memória. *Rust* não usa GC, a memória é gerida através das regras de *ownership* [1].

***Thread*:** É uma unidade de execução que permite executar código de forma concorrente ou paralela. O *Rust* oferece suporte nativo a *threads* através do módulo *std::thread* [10].

***Null Pointer* e *Use-after-Free*:** Erros comuns em *C/C++* ao aceder a memória inválida. Em *Rust*, são prevenidos pelo sistema de tipos e *borrow checker*, aumentando a robustez [11].

***WebAssembly* (WASM):** Formato binário para executar código em ambientes como browsers com um desempenho próximo ao nativo [12].

## 2.2 Estado de Arte

### 2.2.1 Contexto e Características Técnicas

Uma das características mais inovadoras do *Rust* é o seu sistema de gestão de memória sem recurso a um *garbage collector*, que é baseado em dois pilares sendo eles o sistema de *ownership* e o *borrow checker*. Este mecanismo previne que durante a fase de compilação aconteçam erros comuns, como por exemplo o *null pointer dereferencing*, *use-after-free* ou *data races*, garantindo assim a segurança sem comprometer o desempenho [13].

Como linguagem compilada nativamente (*ahead-of-time*), o *Rust* gera executáveis de eficiência comparável à de *C/C++*, sendo ideal para aplicações de baixo nível (sistemas operativos ou drivers), para software empresarial de alta performance ou para ambientes de recursos limitados (sistemas embebidos ou *IoT*) [1].

Além disso, o *Rust* foi desenhado para facilitar a escrita de código concorrente de forma segura. A abordagem *"fearless concurrency"* permite aos programadores criarem aplicações paralelas sem os erros comuns, como ter partes do código a interferirem umas com as outras ou até mesmo a corromperem dados partilhados. O *Rust* verifica automaticamente durante a sua compilação se tudo está seguro evitando assim surpresas indesejadas quando o programa está em execução [14] [15].

### 2.2.2 Comparação com Tecnologias Similares

**Rust vs C/C++** [16]

Prós:

- Segurança de memória garantida pelo sistema de *ownership* e *borrowing*.
- Eliminação de fugas de memória (*memory leaks*) sem necessidade de *garbage collector*.

- Ferramentas modernas como o *Cargo* (gestor de pacotes e sistema de *builds*).
- Suporte nativo a *WebAssembly*.

Contras:

- Curva de aprendizagem mais acentuada devido ao sistema de *ownership*.
- Tempo de compilação mais longo em comparação com *C/C++*.

**Rust vs Go** [17]

Prós:

- Desempenho superior em muitos cenários, graças à ausência de *garbage collector*.
- Controlo mais refinado sobre a alocação de memória.
- Maior segurança na concorrência (Go usa *goroutines*, Rust usa *threads* seguras por design).
- Suporte a *WebAssembly* mais avançado.

Contras:

- *Go* é mais fácil de aprender e adotar.
- Ecossistema de bibliotecas mais vasto para aplicações *web* e distribuídas.

**Rust vs Swift** [18]

Prós:

- Melhor suporte para desenvolvimento de sistemas de baixo nível.
- Compatível com múltiplas plataformas, enquanto o *Swift* está focado no ecossistema *Apple*.
- Comunidade *open-source* mais ampla e diversificada.

Contras:

- *Swift* é mais intuitivo para desenvolvimento de aplicações *iOS/macOS*.
- *Swift* inclui suporte nativo à gestão automática de memória via *ARC* (*contagem automática de referências*).

| **Característica** | **Rust** | **C/C++** | **Go** | **Swift** |
|---|---|---|---|---|
| Compilador Nativo | ✓ | ✓ | ✓ | ✓ |
| Garbage Collector (GC) | ✗ | ✗ | ✓ | △ |
| Segurança de Memória | ✓ | ✗ | △ | △ |
| Concorrência Integrada | ✓ | ✗ | ✓ | ✓ |
| Cross-Platform | ✓ | ✓ | ✓ | △ |

| | | | | |
|---|---|---|---|---|
| Package Manager Oficial | ✓(Cargo) | ✗ | ✓(Go) | ✓(SPM) |
| Suporte a WebAssembly | ✓ | △ | △ | ✗ |
| Ecossistema Open-Source | ✓ | ✓ | ✓ | △ |

**Tabela 1 - Comparação do *Rust* com diferentes tecnologias**

✓ = Suporta/possui nativamente; △ = Suporte parcial ou dependente de bibliotecas externas; ✗ = Não suporta/requer implementação manual.

O *Rust* representa um avanço significativo no desenvolvimento de sistemas seguros e eficientes, reduzindo vulnerabilidades comuns encontradas em *C* e *C++*. No entanto, a sua adoção ainda enfrenta desafios devido à curva de aprendizagem e aos tempos de compilação. A sua natureza *open-source* e a sua rápida expansão na comunidade indicam que o *Rust* continuará a ser uma escolha relevante para os programadores preocupados com a segurança e o desempenho, especialmente em áreas como *Embedded Systems*, *Networking* ou *WebAssembly* [1] [16] [18].

## 3. Perspetivas da tecnologia

### 3.1 Configuração/Instalação (SW/HW)

Para começar a programar em *Rust*, é essencial instalar a ferramenta que gere o compilador, as bibliotecas e os pacotes, chamada *Rustup*. O processo de instalação é simples, mas pode variar dependendo do sistema operativo utilizado. Neste guia, vamos cobrir os passos para instalar o *Rust* tanto no *Windows* quanto no *Linux*, para que você possa configurar o ambiente de desenvolvimento necessário para começar a programar em *Rust*.

A seguir, vamos apresentar os passos necessários para a instalação em ambos os sistemas operativos através de *screenshots* do ecrã que ilustram cada etapa.

#### 3.1.1 Windows

Vamos começar pela instalação em ambiente *Windows*, esteja atento aos *screenshots* apresentados ao longo deste guião [19].




**Figura 1 - *Download* do executável**

Depois de fazer o *download* do executável, basta ir às transferências, e abrir com permissões de administrador para evitar problemas futuros na instalação.

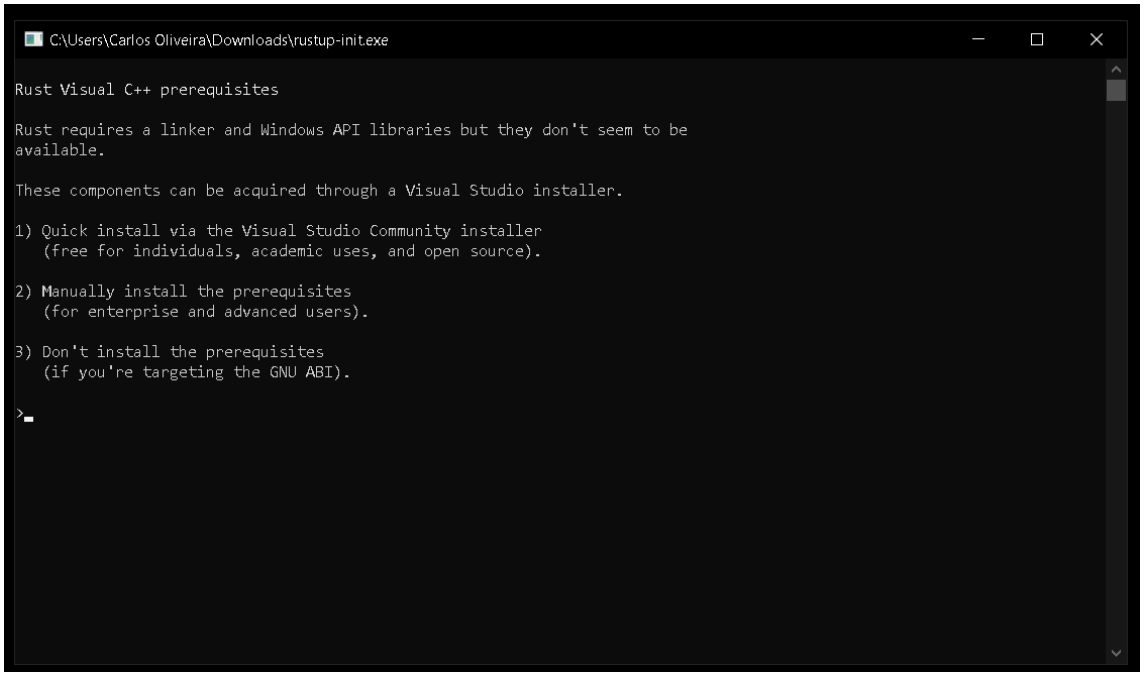



**Figura 2 - Menu 1 de instalação**

Neste primeiro menu de instalação, terá de escolher digitar o número 1 e continuar, instalará automaticamente o *Visual Studio*.

Quando aparecer o seguinte menu, deve selecionar todas as opções.

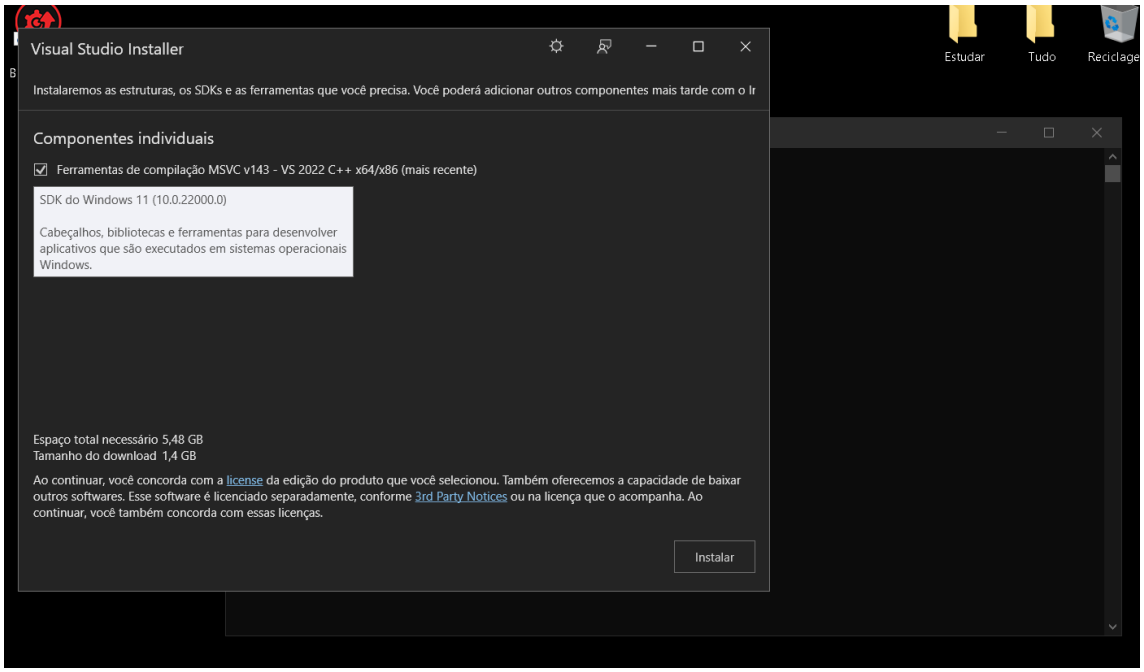



**Figura 3 - Instalação do *Visual Studio***

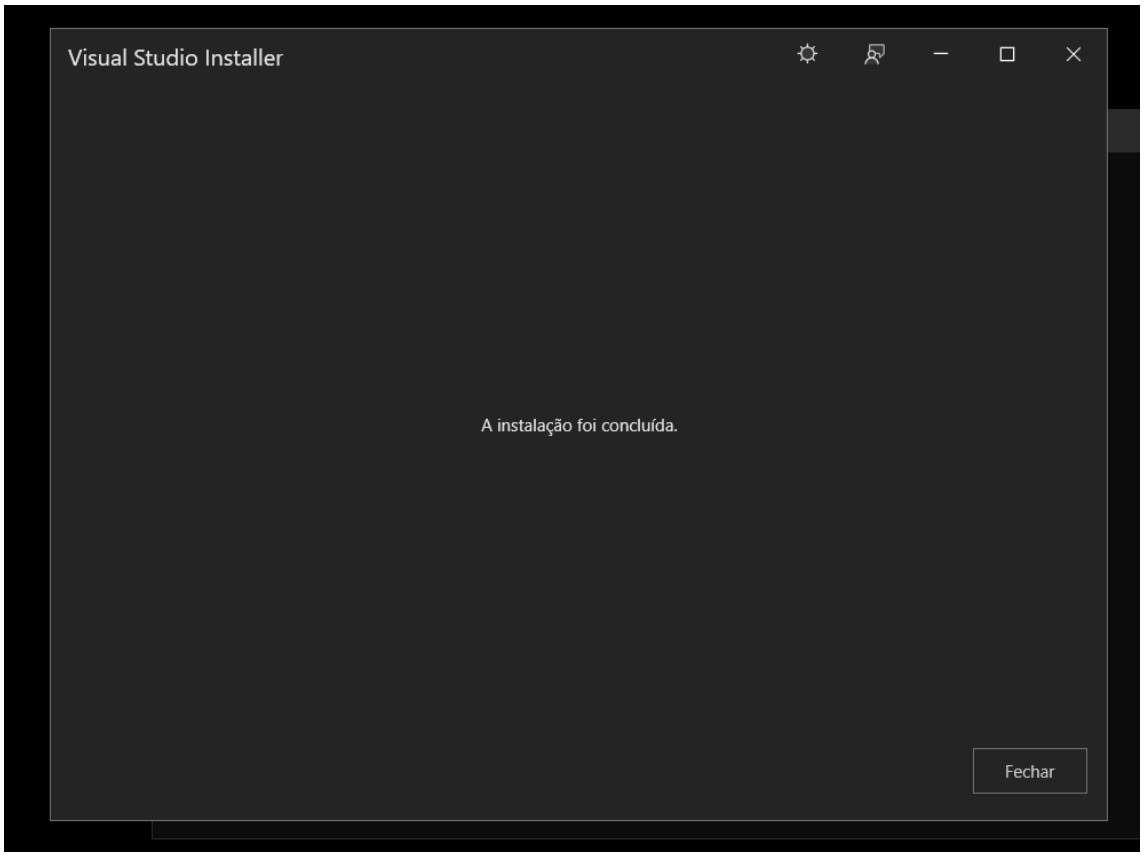



**Figura 4 - *Visual Studio* instalado**

Instalação concluída, prossegue os seguintes passos:

No segundo menu de instalação, basta digitar novamente o número 1 e continuará a instalação do *Rust*.

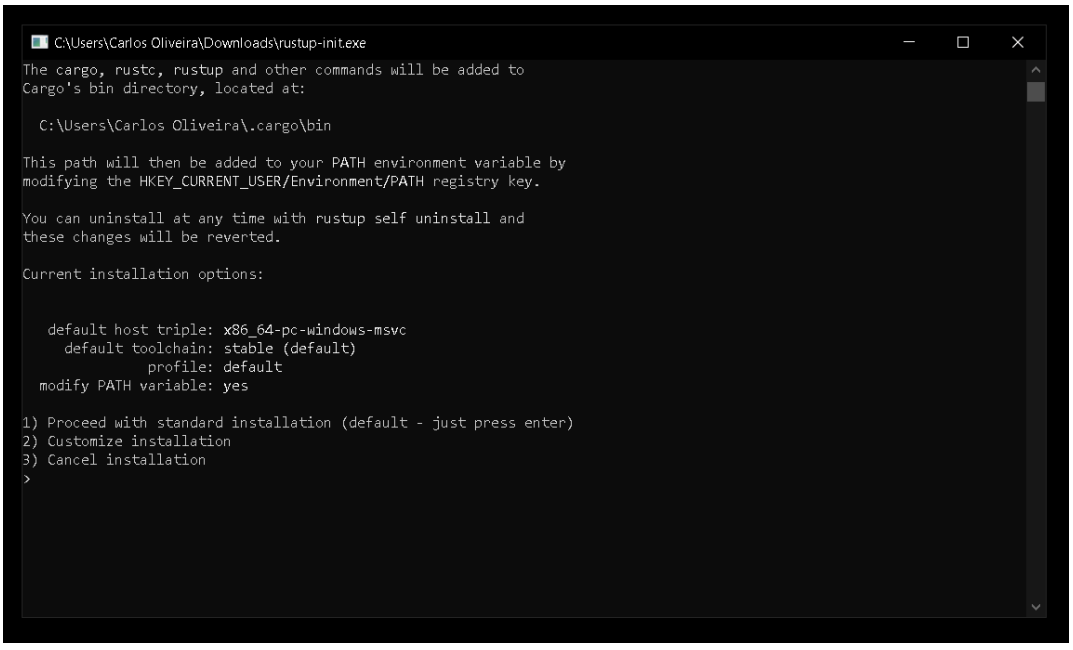



**Figura 5 - Menu 2 de instalação**

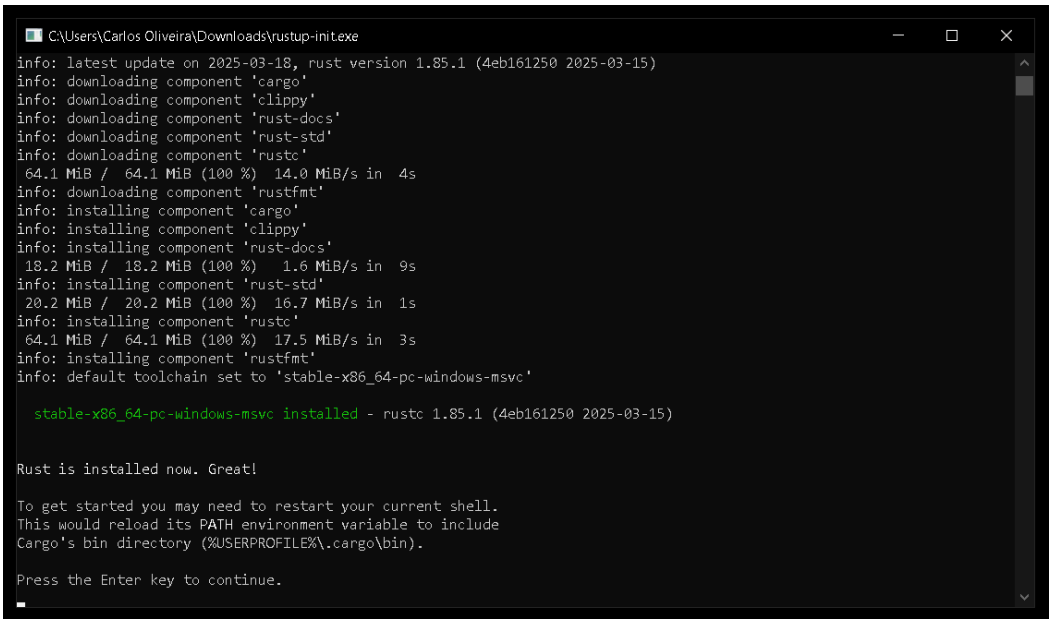



**Figura 6 – Rust instalado**

Instalação do *Rust* finalizada, de seguida, abrir a linha de comandos em modo administrador e digitar o seguinte comando:

```
rustc --version
```

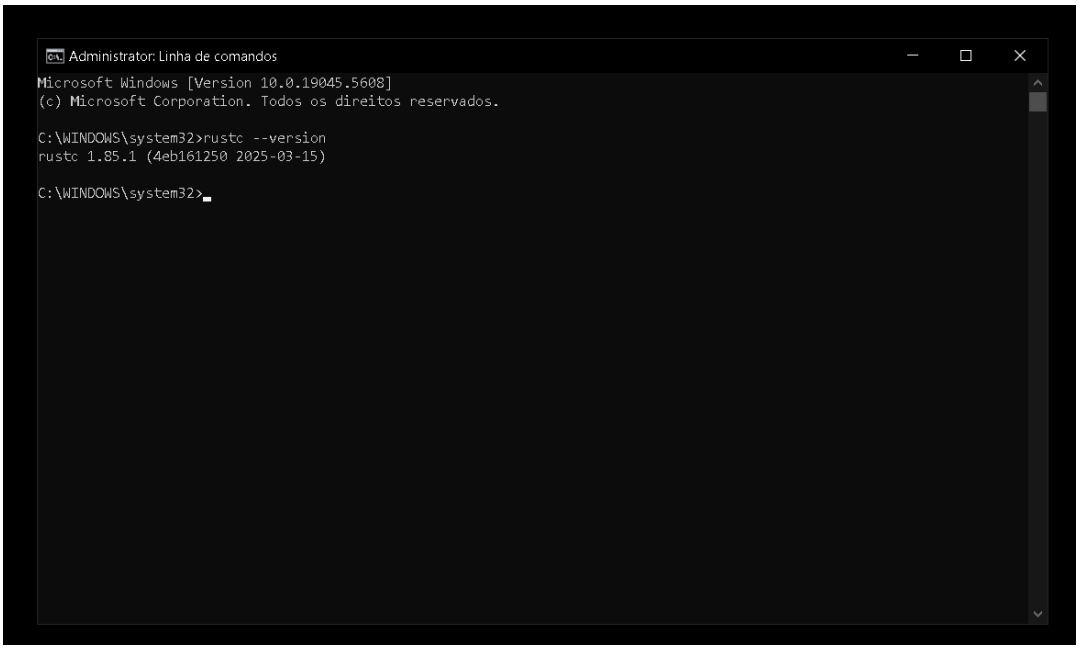



**Figura 7 - Verificar versão do Rust**

A instalação em ambiente *Windows* estará completa, agora é só começar a explorar a linguagem *Rust*.

### 3.1.2 Linux

Para a instalação em ambiente *linux*, começamos por utilizar o comando indicado no site oficial, através do terminal.

```
curl --proto '=https' --tlsv1.2 -sSf https: //sh.rustup.rs | sh
```




**Figura 8-Execução do comando**

Após isso, são apresentadas as seguintes opções. Para este guião foi utilizada a instalação padrão, ou seja, a opção número 1.

```
  /home/pedro/.cargo/bin

This path will then be added to your PATH environment variable by
modifying the profile files located at:

  /home/pedro/.profile
  /home/pedro/.bashrc

You can uninstall at any time with rustup self uninstall and
these changes will be reverted.

Current installation options:


   default host triple: x86_64-unknown-linux-gnu
     default toolchain: stable (default)
               profile: default
  modify PATH variable: yes

1) Proceed with standard installation (default - just press enter)
2) Customize installation
3) Cancel installation
>
```

**Figura 9-Opções de instalação (Prosseguir com a instalação padrão)**

Depois da instalação ter sido terminada, deverá ser apresentada a mensagem "Rust is installed now. Great!". Seguido da confirmação de que a variável de ambiente foi adicionada corretamente.

```
Rust is installed now. Great!

To get started you may need to restart your current shell.
This would reload your PATH environment variable to include
Cargo's bin directory ($HOME/.cargo/bin).

To configure your current shell, you need to source
the corresponding env file under $HOME/.cargo.

This is usually done by running one of the following (note the leading DOT):
. "$HOME/.cargo/env"            # For sh/bash/zsh/ash/dash/pdksh
source "$HOME/.cargo/env.fish"  # For fish
source "$HOME/.cargo/env.nu"    # For nushell
pedro@pedro-Virtual-Machine:~$
```

**Figura 10-Rust instalado**

Versão do compilador *rustc*, usada neste guião.

```
pedro@pedro-Virtual-Machine:~/sistca/Hello_World/src$ rustc --version
rustc 1.85.1 (4eb161250 2025-03-15)
```

**Figura 11-Versão do rustc**

Versão da *toolchain cargo*.

```
pedro@pedro-Virtual-Machine:~/sistca$ cargo --version
cargo 1.85.1 (d73d2caf9 2024-12-31)
```

**Figura 12-Versão do cargo**

### 3.2 Primeiros passos

Vamos aprender como começar a programar em *Rust*, criando e executando um programa simples que imprime "Hello, World!" na tela. O processo será dividido em algumas etapas simples, que você pode seguir facilmente no seu ambiente de desenvolvimento.

**Abrir o Terminal**: Abra a linha de comandos (terminal) com privilégios de administrador. Isso é importante para garantir que você tenha permissão para instalar ou executar os comandos necessários no seu sistema.

**Criar um Novo Projeto em *Rust***: O próximo passo é criar um projeto utilizando o *Cargo*, a ferramenta de gere os pacotes e os *builds* do *Rust*. Para isso, execute o seguinte comando:

```
Cargo new hello_world
```

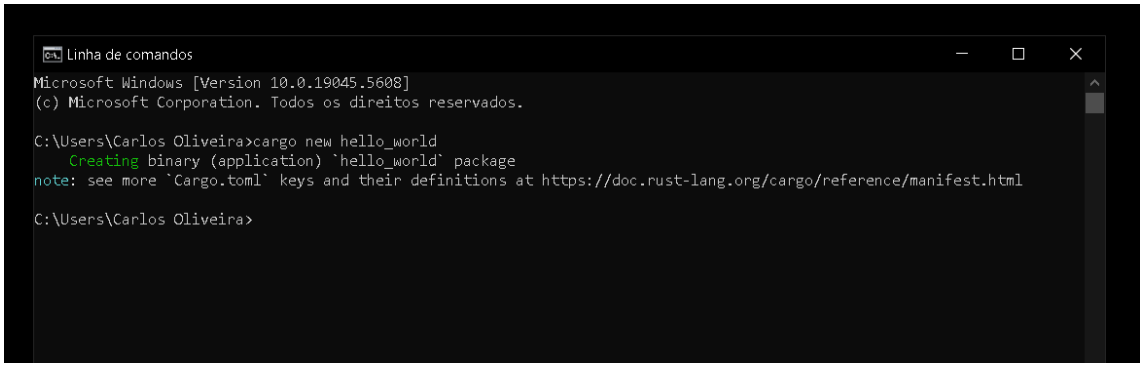



**Figura 13-Criação do projeto *hello_world***

Esse comando cria um diretório chamado ***hello_world*** com a estrutura básica de um projeto *Rust*.

**Entrar no Diretório do Projeto**: Após criar o projeto, entre no diretório gerado com o comando:

```
cd hello_world
```




**Figura 14-Caminho para o projeto**

**Executar o Programa**: Agora, para ver o código em ação, basta usar o *Cargo*. Execute o seguinte comando:

```
cargo run
```

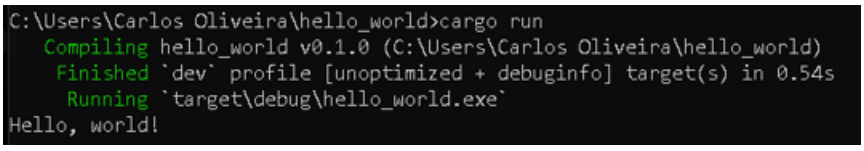



**Figura 15-Execução do projeto**

**Estrutura do código descrito em cima:**

- **fn *main()*:** Define a função principal do programa, onde o código será executado;
- ***println!("Hello, World!")*:** Essa linha imprime a mensagem "Hello, World!" no terminal. A macro ***println!*** é usada para exibir texto na saída padrão.

```rust
fn main() {
    println!("Hello, World!");
}
```

# 4. Tutorial/Funcionalidade

## 4.1Variáveis

### Mutáveis e Imutáveis

As variáveis em *rust* são, por padrão, imutáveis, o que significa que o valor delas não pode ser alterado. Para declarar uma variável, usa-se o prefixo *let*, seguido do nome da variável e do valor [20]. Como se pode observar no exemplo abaixo, este excerto de código vai criar um variável *x*, atribuir-lhe o valor 3 e imprimi-la. Sendo o resultado: "x:3".

```
fn main(){

        let x = 3;

        println!("x:{x}");

}
```

No entanto, caso se tente mudar o valor da variável, como no exemplo em baixo, iria ocorrer o erro da figura 16 [20].

```
fn main(){

        let x = 3;

        x = 6;

}
```

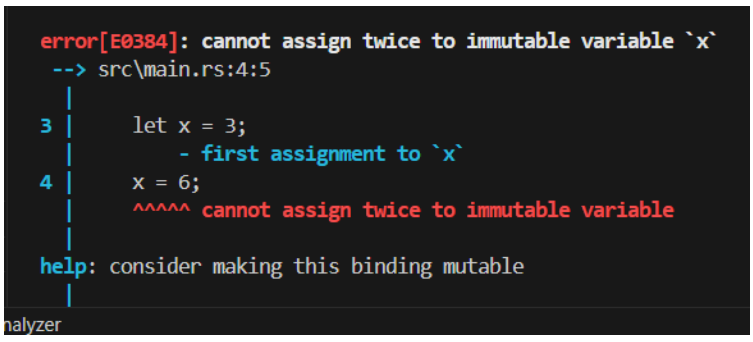



**Figura 16-Tentativa de atribuição de um valor a uma variável imutável**

Isto pode ser resolvido ao adicionar a frente do *let* a palavra *mut*. Isto fará com que a variável criada se torne mutável, e que o seu valor possa ser alterado. Ao utilizar uma variável mutável o erro anterior não ocorre, como se pode ver no exemplo abaixo [20]. O resultado deverá ser "y:5" e na linha em baixo "y:6".

```
fn main(){

        let mut y = 5;

        println!("y:{y}");

        y = 6;

        println!("y:{y}");

}
```

### Constantes

As varáveis constantes são definidas com a palavra-chave *const*, seguido do nome da variável, o tipo e o valor, como no exemplo abaixo [20]. Ao correr o programa deverá imprimir: "PI:3.1415".

```
const PI : i32 = 3.1416;

fn main(){

        println!("PI:{PI}");

}
```

### Shadowing:

Com o *shadowing*, é possivel criar uma espécie de compartimento que não interfere com o *scope* original. Isto é feito ao criar um *inner scope* utilizando chavetas. Como pode ser visto no exemplo abaixo, a nova variável *z* criada a partir do *shadowing* não altera o *outer scope*, ou a variável presente originalmente no *main* [20]. Ao correr, este exemplo o resultado deverá ser "Z (Inner Scope): 15" e "Z(Outer Scope) :10".

```
fn main(){

        let z = 5;

        let z = z + 5;

        {

                let z = z + 5;

                println!("Z (Inner scope): {z}");

        }

        println!("Z (Outer Scope):{z});

}
```

*Shadowing*, é diferente de utilizar uma variável mutável, pois é lançado um erro ao tentar reatribuir o valor sem a palavra-chave *let*. Ao usar a palavra-chave *let*, estamos, essencialmente, a criar uma variável nova. Podendo assim, alterar o tipo do valor que a variável recebe [20]. Como é possível ver no exemplo abaixo, inicialmente a variável recebe uma *string* com espaços, mas ao tentar atribuir um valor inteiro com *spaces.len()*, sem a palavra-chave *let*, o programa lança o erro apresentado na figura 17.

```
fn main(){

        let mut spaces = ” “;

        spaces = spaces.len();

}
```

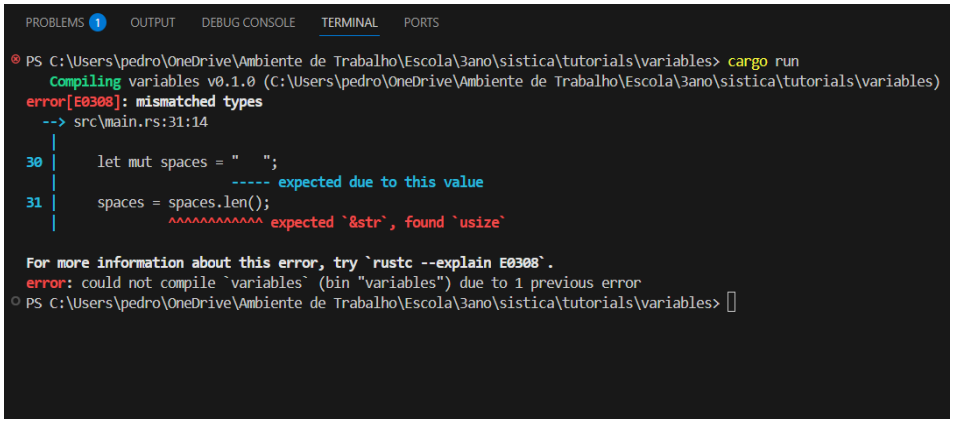



**Figura 17-Atribuição de um valor com o tipo diferente ao esperado a uma variável mutável**

No entanto, ao utilizar a palavra-chave *let* como no exemplo em baixo, o erro não ocorre, imprimindo “spaces: 3” [20].

```
fn main(){

        let spaces = “  “;

        let spaces = spaces.len();

        println!(“spaces: {spaces}”);

}
```

### 4.2 Tipos de Dados

Neste tópico será explorado os vários tipos de dados existentes na linguagem *Rust* [21].

Tipos de dados que existem:

- Tipos escalares
- Tipos compostos

**Tipos escalares**

**Inteiros**

Os inteiros são números sem parte decimal. Em *Rust*, podem ser:

***Signed* (podem ser negativos e positivos):**

- i8 → de -128 a 127
- i16 → de -32.768 a 32.767
- i32 → de -2.147.483.648 a 2.147.483.647
- i64, i128 → números ainda maiores
- isize → depende da arquitetura do sistema (32 ou 64 bits)

***Unsigned* (apenas positivos):**

- u8 → de 0 a 255
- u16 → de 0 a 65.535
- u32 → de 0 a 4.294.967.295
- u64, u128 → números maiores
- usize → depende da arquitetura do sistema (32 ou 64 bits)

Neste exemplo, esta demonstrado como se declara o tipo de dados inteiro.

```
fn main(){

        let idade: u8 = 25; // so aceita valores de 0 a 255

        let saldo: i32 = -500; // aceita valores positivos e negativos

}
```

### Floating Point

Os números de pontos flutuantes são aqueles com casas decimais, como 3,14 ou -0,75.

Em *Rust* tem **dois tipos principais** para este tipo de números:

- f32 → ocupa **32 bits** (menos preciso)
- f64 → ocupa **64 bits** (mais preciso e é o **padrão** em *Rust*)

Ambos os tipos podem representar valores negativos e positivos (são assinados).

Neste exemplo, esta demonstrado como se declara o tipo de dados ponto flutuante.

```
fn main(){

        let x = 5.0;      // f64 (por padrão)

        let y: f32 = 3.70;   // f32 (definido com anotação de tipo)

}
```

### *Boolean*

Em *Rust* o tipo *boolean* representa apenas dois valores possíveis:

- *true* → verdadeiro
- *false* → falso

O tipo boolean é chamado ***bool*** e ocupa **1 *byte*** na memória.

Neste exemplo, esta demonstrado como se declara o tipo de dados *boolean*(*bool*).

```
fn main(){

        let t = true;          // valor -> verdadeiro

        let f: bool = false;     // valor -> falso (com anotação de tipo)

}
```

### Char

Em *Rust*, o tipo *char* serve para representar um único caractere. Este tipo suporta muito mais do que apenas letras e números — também permite símbolos, letras acentuadas, *emojis*, e caracteres de outros idiomas como chinês ou japonês,etc...

- Os caracteres são definidos entre **aspas simples** (' '). Diferente das *strings*, que usam **aspas duplas** (" ").
- Cada ***char*** ocupa 4 *bytes*, pois segue o padrão *Unicode*, podendo representar milhares de símbolos diferentes.

Neste exemplo, esta demonstrado como se declara o tipo de caractere(*char*).

```
fn main(){

        let letra = 'b';

        let simbolo = '✈';

        let emoji = ' ⚽ ';

        println!("Letra: {letra}");

        println!("Símbolo: {simbolo}");

        println!("Emoji: {emoji}");

}
```

## Tipos compostos

### Tuplo

Em *Rust* um tuplo é uma estrutura que agrupa vários valores, mesmo que sejam de tipos diferentes. A sua dimensão é fixa e depois de criado, não se pode adicionar ou remover elementos.

Exemplo de como criar um tuplo em *Rust*:

```
fn main(){

let tuplo: (i32, f64, u8) = (580, 6.7, 2); // um tuplo com três tipos diferentes

}
```

Também é possível ter acesso diretamente a um elemento do tuplo pelo seu índice (começando em 0):

```
fn main(){

  let x = (500, 6.4, 1);

  let primeiro = x.0;

  let segundo = x.1;

  let terceiro = x.2;

}
```

Existe ainda o tuplo vázio, chamado *unit*, escrito como () e representa "nada" ou "sem valor".

**Array**

Em *Rust* um *array* é uma coleção de elementos todos do mesmo tipo e com tamanho fixo. É útil quando sabes à partida quantos elementos vais ter.

Exemplo de como criar um *array*:

```
fn main(){

        let numeros = [1, 2, 3, 4, 5]; // array de 5 inteiros

}
```

Como declarar um *array* de um certo tipo, desta forma:

```
fn main(){

        let a: [i32; 5] = [1, 2, 3, 4, 5];

}
```

Como criar um *array* com valores repetidos, desta forma:

```
fn main(){

        let a = [3; 5]; // equivale a [3, 3, 3, 3, 3]

}
```

Para ter acesso a elementos de um *array*, é utilizado índices, desta forma:

```
fn main(){

   let a = [1, 2, 3, 4, 5];

   let primeiro = a[0];

   let segundo = a[1];

}
```

Tem de ter cuidado caso tente aceder a uma posição fora do limite do *array*, porque o programa vai gerar um erro e parar. Pois isto faz parte do sistema de segurança do *Rust*.

### 4.3 Funções

As funções são um componente central no *Rust*, destacando-se a função *main* como o ponto de entrada da maioria dos programas.

Por convenção, a linguagem adota o *snake case* para nomes de funções e variáveis, a onde todas as letras são minúsculas e os *underscores* (_) separam palavras compostas (ex.: *calcular_media*). Um exemplo de implementação é dado pelo codigo abaixo [22]:

```
fn main() {

   println!("Hello, world!");

   another_function();

}

fn another_function() {

   println!("Another function.");

}
```

Uma função é definida através da palavra-chave *fn*, seguida do nome da função e de parênteses "()". As chavetas "{}" delimitam o início e o fim do corpo da função, indicando ao compilador onde o código da mesma se encontra. Para invocar uma função, basta utilizar o seu nome seguido de parênteses [22].

No exemplo apresentado, a função *another_function()* é chamada dentro da função *main()*, uma vez que está definida no mesmo âmbito. É irrelevante se a função é declarada antes ou depois da *main()* no código-fonte, o *Rust* apenas exige que a função seja acessível ao local onde é chamada.

Após executarmos o código teremos algo como o que está representado na figura 12.

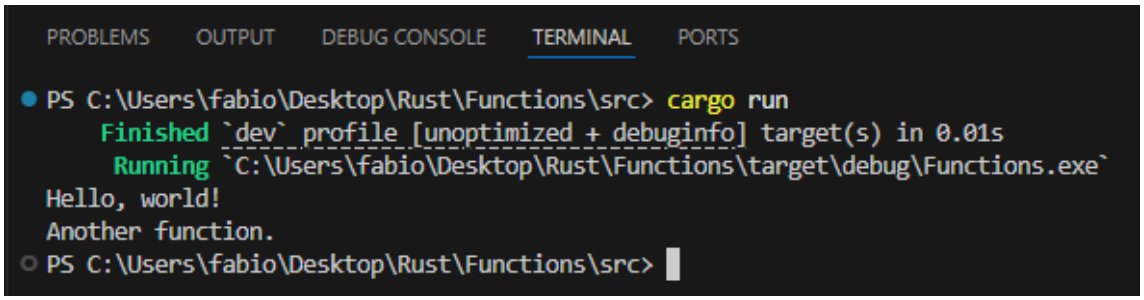



**Figura 18 - Demonstra a ordem sequencial das instruções da função *main()***

A execução das instruções ocorre pela ordem em que são definidas na função *main()*. Assim, a mensagem "Hello, world!" é impressa primeiro, seguida da mensagem "Another function.", pois a função *another_function()* está depois do "println!("Hello, world!")".

### 4.3.1 Funções com parâmetros

Nas funções podemos incluir variáveis especiais chamadas de parâmetros, que fazem parte da assinatura (*signature*) da função. Quando invocamos essa função, damos valores concretos a esses parâmetros, chamados de argumentos [22].

```
fn main() {

  labeled_measurement(5, 'h');

}

fn labeled_measurement(value: i32, unit: char) {

  println!("The measurement is: {value}{unit}");

}
```

Neste exemplo definimos a função *labeled_measurement()* com dois parâmetros, separados por uma vírgula ",", chamados de *value* e *unit*. Quando executado o código o resultado obtido, figura 13, é "The measurement is: 5h", isto acontece, porque quando é chamada a função *labeled_measurement()* é passado dentro dela os argumentos "5, 'h'".

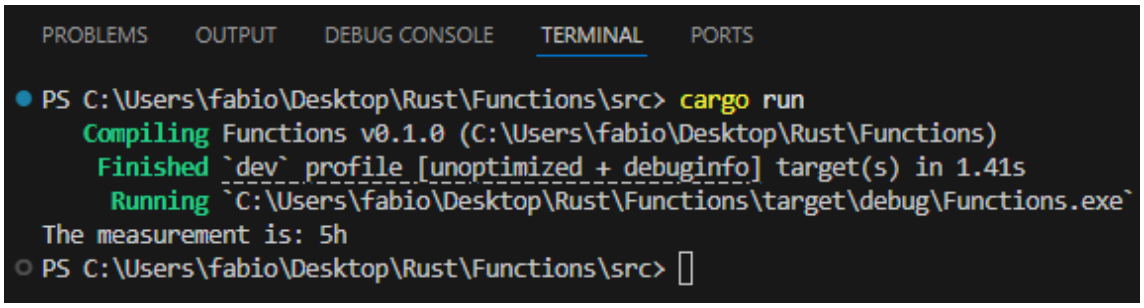



**Figura 19 - Resultado da função *labeled_measurement()***

Nota: Obrigatoriamente declaramos o tipo de cada parâmetro na assinatura da função. Isto permite que o compilador compreenda qual é o tipo de valor esperável em cada variável, tornando assim desnecessárias anotações adicionais noutras partes do código. Permite também gerar mensagens de erro mais precisas quando alguma coisa não corresponder ao que a função exige.

### 4.3.2 Funções com Valores de Retorno

Em *Rust*, as funções podem devolver resultados ao código que as executa. Embora não seja necessário atribuir nomes a esses valores, o tipo de retorno (*return*) deve ser definido após uma seta (->). A particularidade do *Rust* é que o valor devolvido corresponde sempre à última expressão avaliada no corpo da função [22].

```
fn main() {

   let x = five();

   println!("The value of x is: {x}");

}

fn five() -> i32 {

   5  // Expressão final = valor de retorno

}
```

A função *five()* não tem parâmetros, chamadas, ou variáveis tem apenas o valor 5. O tipo i32 indica que devolve um inteiro de 32 bits [22].

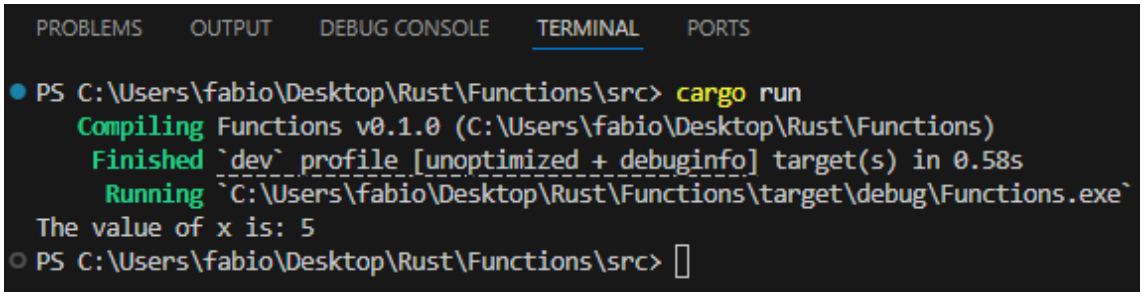



**Figura 20 - Resultado da main()**

Quando executado o código o valor devolvido é 5, figura 14, isto acontece porque se esta a usar o valor de retorno da função *five()* para inicializar a variável x (encontra-se na função *main()*).

Nota: É possivel usar return para sair antecipadamente, mas na maioria dos casos, o resultado é implícito. A ausência de ";" no 5 é intencional, pois em *Rust* as expressões sem ";" são definidas como retorno.

### 4.4 Control Flow

A capacidade de executar código dependendo de uma condição, seja ela verdadeira ou falsa, é chamada de *control flow*. O controlo de fluxo é uma das funcionalidades mais básicas da maioria das linguagens de programação, não sendo diferente no *Rust*, estando presente em condições *if* e ciclos, *for* ou *while* [23].

**Expressões *if***

Uma condição *if* é criada utilizando a palavra-chave *if*, seguido de uma expressão. Se a condição for verdadeira, irá ser executado o bloco de código logo abaixo da verificação da condição. Caso a condição seja falsa, irá ser executado o bloco de código que está dentro da palavra-chave *else* [23]. Como se pode verificar no excerto de código em baixo:

```
fn main() {

   let num = 2;

   if num % 2 == 0 {

      println!("par");

   }else{

      println!("impar");

   }

}
```

As condições *if* só funcionam quando o seu resultado é um booleano [23]. Por exemplo, no código em baixo, foi criada a seguinte condição:

```
fn main() {

   let number = 3;

   if number {

      println!("number was three");

   }

}
```

Como número não é do tipo booleano irá lançar o erro da figura 21:

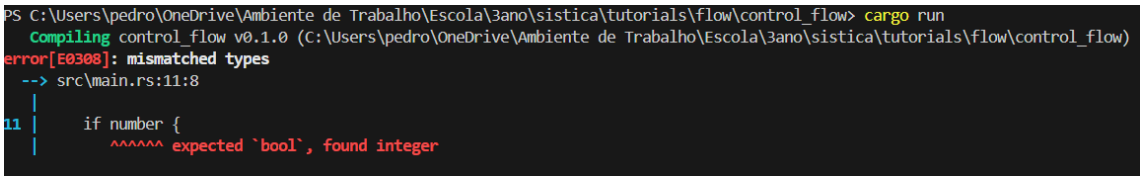



**Figura 21-Condição com um tipo diferente de bool**

É possível, também encadear múltiplos *if* utilizando a palavra-chave *else if* [23]. Neste exemplo, o compilador irá verificar cada condição e executar a primeira condição que se encontra verdadeira.

```
fn main(){

   let number = 2;

   if number == 0{

      println!("É o numero 0");

   }else if number == 1{

      println!("É o numero 1");

   }else if number == 2{

      println!("É o numero 2");

   }

}
```

É também possível utilizar condições *if* em conjunto com a palavra-chave *let*, na criação de uma variável [23]. No exemplo abaixo, estamos a verificar se a condição é verdadeira, se for verdadeira a variável *num* vai receber o número 5, caso contrário irá receber o número 6. É importante mencionar, que os valores que a variável pode receber tem de pertencer ao mesmo tipo, caso contrário, irá ser lançado o erro da figura 22.

```
fn main() {

   let condition = true;



   let num = if condition {5} else {6};



   println!("num: {num}");

}
```

```
PS C:\Users\pedro\OneDrive\Ambiente de Trabalho\Escola\3ano\sistica\tutorials\flow\control_flow> cargo run
   Compiling control_flow v0.1.0 (C:\Users\pedro\OneDrive\Ambiente de Trabalho\Escola\3ano\sistica\tutorials\flow\control_flow)
error[E0308]: `if` and `else` have incompatible types
 --> src\main.rs:4:38
  |
4 |     let num = if condition {5} else {"six"};
  |                            -        ^^^^^ expected integer, found `&str`
  |                            |
  |                            expected because of this

For more information about this error, try `rustc --explain E0308`.
```

**Figura 22-O tipo variável da condição alternativa é diferente da primeira**

## Ciclos

É possível criar um ciclo infinito utilizando a palavra-chave *loop* [23]. Como não tem condição de paragem, é necessário terminar manualmente o programa utilizando o atalho "ctrl + c".

```
fn main(){

        loop{

                println!("Again!");

        }

}
```

Assim como com as condições *if*, também é possível utilizar ciclos para atribuir valores a variáveis. Isto é feito, ao utilizar a palavra-chave *let*, seguido do nome da variável e de um ciclo, como apresentado no exemplo em baixo, desta vez existe uma condição de paragem, sendo esta, o valor da variável *counter* ser igual a 10; quando a condição de paragem é atingida, o ciclo é interrompido com a palavra-chave *break* [23]. Neste exemplo, a variável *result* irá ficar com o valor da variável *counter*, ou seja, 20.

```
fn main() {

  let mut counter = 0;



  let result = loop {

    counter += 1;



    if counter == 10 {

      break counter * 2;

    }

  };

  println!("The result is {result}");

}
```

Em *Rust*, existe um sistema de etiquetas, com o intuito de identifcar ciclos dentro de outros ciclos, isto é feito ao utilizar " ' " seguido do nome do ciclo e da palavra-chave *loop*. Desta forma, é possivel dentro de outros ciclos utilizar palavra-chaves relacionadas a ciclos dentro de outros ciclos [23]. A sintaxe para que isso aconteca é a seguinte: " ` " seguido do nome do ciclo e da palavra-chave que se desja utilizar. No exemplo abaixo, temos dois ciclos, o primeiro tendo uma etiqueta com o nome

*counting_up* e uma dentro dessa sem etiqueta. O primeiro ciclo imprime o valor atual da variável *count* e cria a variável *remaining* com o valor 10, de seguida, entra no segundo ciclo, que imprime o valor da variável *remaining* e fica dentro desse ciclo até o valor da variável *remaining* ser igual a 9, ou caso o valor da variavel *count* seja igual a 2, termina o ciclo *counting_up* onde o ciclo sem nome está instalado. Como o programa só termina o ciclo principal, *counting_up* quando a variavel *count* é 2 o programa termina com a impressão de "End Count = 2".

```
fn main() {

    let mut count = 0;

    'counting_up: loop {

        println!("count = {count}");

        let mut remaining = 10;



        loop {

            println!("remaining = {remaining}");

            if remaining == 9 {

                break;

            }

            if count == 2 {

                break 'counting_up;

            }

            remaining -= 1;

        }



        count += 1;

    }

    println!("End count = {count}");

}
```

Também é possivel utilizar ciclos condicionais, também chamados de ciclos *while* [23]. Isto é feito ao utilizar a *palavra-chave while* seguido de uma condição como no exemplo abaixo:

```
fn main() {

   let mut number = 3;



   while number != 0 {

      println!("{number}!");



      number -= 1;

   }

}
```

Além dos ciclos anteriores, existe também o ciclo *for*. Este é utilizado para percorrer elementos numa *array*, isto é feito ao utilizar a palavra-chave *for*, seguido de uma variável para armazenar o valor atual da *array* onde o indice está a apontar, a palavra-chave *in* e por fim, o nome da *array*, como exemplificado em baixo [23]. Ao executar este codigo deverá imprimir o valor de cada um dos elementos presentes na *array*.

```
fn main() {

   let a = [10, 20, 30, 40, 50];



   for element in a {

      println!("the value is: {element}");

   }

}
```

### 4.5 Borrowing

*Borrowing* é a solução de *Rust* para gerir memória e é o aspeto mais único na linguagem.

Em *Rust*, cada variável tem um e um só dono que quando acaba de utilizar uma variável a memória dela é libertada. Uma variável acaba de ser utilizada quando todas as referências que foram "emprestadas" foram devolvidas e o dono a descarta. Tentar usar a referência de uma variavel já destruida é um erro comum, um exemplo disso seria o seguinte [6]:

```
fn main(){

  {

    let x = 1;

  }

  let y = &x;

}
```

O código acima não compila com o seguinte erro:

```
Errors

Exited with status 101

Standard Error

   Compiling playground v0.0.1 (/playground)
error[E0425]: cannot find value `x` in this scope
 --> src/main.rs:5:14
  |
5 |     let y = &x;
  |             ^
  |
help: the binding `x` is available in a different scope in the same function
 --> src/main.rs:3:13
  |
3 |         let x = 1;
  |             ^

For more information about this error, try `rustc --explain E0425`.
error: could not compile `playground` (bin "playground") due to 1 previous error

Standard Output
```

**Figura 23 - Referência inválida**

Que nos diz que a variável y está a tentar usar uma referência que já foi destruida, ou neste caso apenas está presente numa *scope* diferente.

Uma forma de visualizar como é que vários endereços que levam à mesma memória é a seguinte:

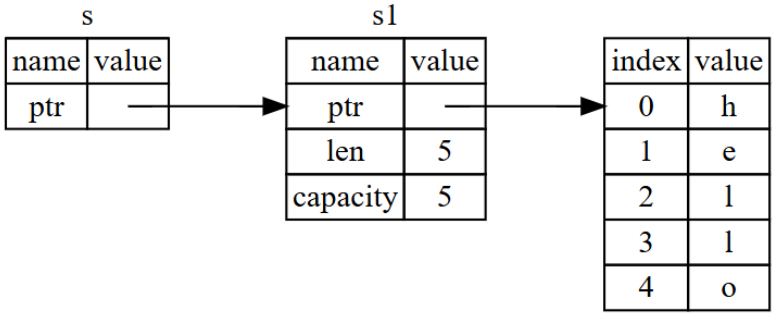



**Figura 24 - Visualização de memória**

A figura 24, representa como está ordenada a memória no seguinte código:

```
fn main() {

   let s1 = String::from("hello");

   let len = calculate_length(&s1);

   println!("The length of '{s1}' is {len}.");

}

fn calculate_length(s: &String) -> usize {

   s.len()

}
```

As duas variáveis (s e s1) apontam para endereços de memória diferentes, mas querem aceder à mesma memória enquanto são de diferentes *scopes*, neste programa isso não produz nenhum problema porque é destruído e não é utilizado a seguir, mas isso só acontece porque o tempo de vida ou *lifetime* dessa variável é conhecido. O tópico de *lifetimes,* por norma, é mais importante quando são utilizados *structs* e isso vai ser abordado na próxima secção [24].

### 4.6 Structs, Enums e Lifetimes

Uma estrutura ou *struct* é uma forma de agrupar diferentes variáveis de vários tipos diferentes. Um exemplo disso:

```
struct User {

   active: bool,

   username: String,

   email: String,

   sign_in_count: u64,

}
```

Enquanto que um *enum* enumera diferentes opções de valores possíveis. Um exemplo disso:

```
enum IpAddrKind {

   V4,

   V6,

}
```

Os campos de um *enum* podem tomar várias formas, por exemplo:

```
enum Message {
    Quit,
    Move { x: i32, y: i32 },
    Write(String),
    ChangeColor(i32, i32, i32),
}
```

*Quit* não tem dados associados.

*Move* tem dois campos com nomes x e y que são números inteiros de 32bit.

*Write* tem um campo sem nome que é uma *String*.

*ChangeColor* tem três campos sem nome que são números inteiros de 32bit [25].

### 4.7 Threads

*Threads* permitem a execução de múltiplas tarefas simultanêamente utilizando mais eficientemente os núcleos do processador. A maior vantagem de usar *threads* em *Rust* é a segurança de partilha de memória partilhada entre várias *threads* que em outras linguagens, como *C/C++*, podem causar *data races* que ocorrem quando, por exemplo, duas *threads* tentam ler e escrever no mesmo endereço de memória causando comportamento inesperado. Este problema não ocorre em *Rust* porque somos forçados a utilizar *Arc<T>* para a partilha de endereços entre *threads* e para escrever em memória é necessário usar *Mutex<T>* que bloqueia o acesso a outras *threads* se essa memória estiver a ser utilizada. [15]

Em *Rust* para criar uma thread utilizamos *thread::spawn* da biblioteca *standard* (std), por exemplo [26] [27]:

```
use std::thread;

fn main() {
    let handle = thread::spawn(|| {
        for i in 1..5 {
            println!("Mensagem da thread filha: {}", i);
        }
    });
    for i in 1..5 {
        println!("Mensagem da thread principal: {}", i);
```

```
    }

    handle.join().unwrap();

}
```

No código acima a função *spawn()* retorna uma *handle* que nos fornecesse o método *join()* que faz com que a thread "mãe" espere que a execução dos "filhos" acabe. O código acima retorna o seguinte:

```
Mensagem da thread principal: 1
Mensagem da thread principal: 2
Mensagem da thread principal: 3
Mensagem da thread principal: 4
Mensagem da thread filha: 1
Mensagem da thread filha: 2
Mensagem da thread filha: 3
Mensagem da thread filha: 4
```

**Figura 25 - *Output* código *threads***

Como foi mencionado anteriormente, é necessário utilizar *Arc<T>* e *Mutex<T>* para garantir a modificação e leitura segura de memória partilhada entre *threads*, esses componentes podem ser utilizados da seguinte forma:

```
use std::sync::{Arc, Mutex};

use std::thread;

fn main() {

    let counter = Arc::new(Mutex::new(0));

    let mut handles = vec![];

    for _ in 0..10 {

        let counter = Arc::clone(&counter);

        let handle = thread::spawn(move || {

            let mut num = counter.lock().unwrap();

            *num += 1;

        });

        handles.push(handle);

    }
```

```
    for handle in handles {
        handle.join().unwrap();
    }
    println!("Resultado final: {}", *counter.lock().unwrap());
}
```

Este código criar uma variável *counter* que cria uma referência partilhada de um *Mutex* que bloqueia o acesso ao valor 0. Após estar definida essa variável cada *thread* pode clonar a variável para ter acesso à memória partilhada mas apenas quando utilizam *lock()* que é um método do *Mutex* que só dá acesso ao valor 0 quando mais nenhuma *thread* lhe está a aceder e de seguida incrementa esse valor. Quando uma *thread* acaba de utilizar a memória correspondente ao *counter* ela é desbloqueada e uma outra *thread* continua a sua execução. Este processo é seguro mas tem um custo de desempenho que pode ou não ser significativo consoante a complexidade da tarefa a executar. O resultado final da execução do código é 10 porque 10 *threads* são criadas e cada uma incrementa o mesmo endereço de memória uma vez.

## 5. Exercício(s)

### 5.1 Problema A

Neste exercício, será necessário aplicar conhecimentos sobre manipulação de *strings* e tipos numéricos em *Rust*. Crie um programa que leia um número inteiro positivo fornecido pelo utilizador e, de seguida, conte quantos dígitos desse número são os pares e quantos são os ímpares. O programa deverá garantir que a entrada é válida (ou seja, um número inteiro positivo).

Dica: Utilize a função *.parse()* para converter a entrada do utilizador para um número inteiro, e o método *.chars()* para iterar sobre os dígitos do número.

#### 5.1.1 (re)solução A

```
use std::io;

fn main() {
    let mut input = String::new();
    println!("Insere um número inteiro positivo:");
    io::stdin()
        .read_line(&mut input)
        .expect("Erro ao ler a linha");
    let input = input.trim();
    // Verifica se o input é um número válido
```

```
    let numero: u64 = match input.parse() {
        Ok(n) => n,
        Err(_) => {
            println!("Valor inválido. Por favor insere um número inteiro positivo.");
            return;
        }
    };
    let mut pares = 0;
    let mut impares = 0;
    for ch in input.chars() {
        if let Some(digito) = ch.to_digit(10) {
            if digito % 2 == 0 {
                pares += 1;
            } else {
                impares += 1;
            }
        }
    }
    println!("O número tem {} dígito(s) par(es) e {} dígito(s) ímpar(es).", pares, impares);
}
```

### 5.2 Problema B

Neste exercício, vai ser necessário aplicar conhecimentos sobre vetores, ciclos e manipulação de *strings*. Crie um programa que implemente um sistema de gestão de inventário, onde o utilizador pode adicionar, listar e procurar itens.

Dica: Utilize um *loop* para permitir que o programa continue a correr até que o utilizador decida sair. Para procurar os itens, pode utilizar o método *.contains()* em conjunto com *.to_lowercase()* para tornar a pesquisa insensível a maiúsculas/minúsculas.

### 5.2.1 (re)solução B

```
use std::io;

fn main() {
  let mut inventario: Vec<String> = Vec::new();
  loop {
    println!("\n--- Menu do Inventário ---");
    println!("1. Adicionar item");
    println!("2. Listar itens");
    println!("3. Procurar item");
    println!("4. Sair");
    println!("Escolha uma opção:");
    let mut escolha = String::new();
    io::stdin().read_line(&mut escolha).expect("Erro ao ler opção");
    match escolha.trim() {
      "1" => {
        println!("Digite o nome do item a adicionar:");
        let mut item = String::new();
        io::stdin().read_line(&mut item).expect("Erro ao ler item");
        let item = item.trim().to_string();
        if !item.is_empty() {
          inventario.push(item);
          println!("Item adicionado com sucesso!");
        } else {
          println!("Item inválido.");
        }
      }
      "2" => {
```

```
            println!("\n--- Lista de Itens ---");
            if inventario.is_empty() {
                println!("O inventário está vazio.");
            } else {
                for (i, item) in inventario.iter().enumerate() {
                    println!("{}. {}", i + 1, item);
                }
            }
        }
        "3" => {
            println!("Digite o nome do item a procurar:");
            let mut procura = String::new();
            io::stdin().read_line(&mut procura).expect("Erro ao ler item");
            let procura = procura.trim().to_lowercase();
            let encontrados: Vec<&String> = inventario
                .iter()
                .filter(|item| item.to_lowercase().contains(&procura))
                .collect();
            if encontrados.is_empty() {
                println!("Item não encontrado.");
            } else {
                println!("Item(s) encontrado(s):");
                for item in encontrados {
                    println!("- {}", item);
                }
            }
        }
```

```
        "4" => {
            println!("A sair do programa...");
            break;
        }
        _ => {
            println!("Opção inválida. Tente novamente.");
        }
    }
  }
}
```

### 5.3 Problema C

Neste exercício vai ser necessário aplicar conhecimentos sobre *closures*, *borrowing* e iteradores. Crie um programa que declare um vetor de números inteiros e de seguida uma função que encontre todos os números par e multiplique cada um por 2 e no final imprima esses valores todos.

Dica: Utiliza as funções *.filter()* e *.map()* para filtrar os números pares e multiplicar cada elemento por dois com uma *closure*.

#### 5.3.1 Resolução C

```
fn main() {
  let numbers = vec![1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12];


  // Closure para multiplicar por 2
  let multiply_by_two = |num: i32| -> i32 {
    num * 2
  };


  // Filtra os números pares e aplica a closure para multiplicar por 2
  let processed_numbers: Vec<i32> = numbers
```

```
    .iter()
    .filter(|&num| num % 2 == 0) // Filtra números pares
    .map(|&num| multiply_by_two(num)) // Aplica a multiplicação
    .collect();

  println!("Números pares multiplicados por 2: {:?}", processed_numbers);
}
```

### 5.4 Problema D

Este exercício vai aplicar conhecimentos de *threads*. Crie um programa que em simultâneo calcule n (pode ser introduzido pelo utilizador ou não) números *fibonacci* e calcule o factorial de n.

#### 5.4.1 Resolução D

Funções para calcular n números fibonacci e factorial de n:

```
use std::thread;

fn fibonacci(n: usize) -> Vec<u64> {
  let mut fibs = vec![0, 1];
  for i in 2..n {
    let next = fibs[i - 1] + fibs[i - 2];
    fibs.push(next);
  }
  fibs.truncate(n); // In case n < 2
  fibs
}

fn factorial(n: u64) -> u128 {
  (1..=n).map(|x| x as u128).product()
}
```

```
fn main() {
    let fib_n = 30;
    let fact_n = 20;

    println!("Starting calculations...");

    let fib_thread = thread::spawn(move || {
        let result = fibonacci(fib_n);
        println!("[Fibonacci] First {} Fibonacci numbers:", fib_n);
        println!("{:?}", result);
    });

    let factorial_thread = thread::spawn(move || {
        let result = factorial(fact_n);
        println!("[Factorial] {}! = {}", fact_n, result);
    });
    fib_thread.join().unwrap();
    factorial_thread.join().unwrap();

    println!("Done!");
}
```

## 6. Exercícios Complementares

Os seguintes exercícios vão complementar os anteriores, vão ser apresentados excertos de código, dentro dos mesmos tópicos já abordados, com erros que deveram ser encontrados e arranjados.

### 6.1 Problema A

Neste problema é necessário entender os básicos do sistema de *borrowing* do *Rust*. Corriga o erro no seguinte código:

```
fn get_char(data: String) -> char {

   data.chars().last().unwrap()

}



fn string_uppercase(mut data: &String) {

   data = data.to_uppercase();



   println!("{data}");

}



fn main() {

   let data = "Rust is great!".to_string();



   get_char(data);



   string_uppercase(&data);

}
```

Dica: Apenas é preciso adicionar/remover &.

### 6.1.1 Resolução A

```
fn get_char(data: &String) -> char {
   data.chars().last().unwrap()
}

fn string_uppercase(mut data: String) {
   data = data.to_uppercase();
   println!("{data}");
}
fn main() {
   let data = "Funciona!".to_string();
   get_char(&data);
   string_uppercase(data);
}
```

## 6.2 Problema B

Este problema engloba *borrowing* e *threads* e apresenta um grau de dificuldade consideravelmente mais alto. Corriga o(s) erro(s) presente(s) neste código:

```
use std::thread;

fn main() {
   let numbers = vec![1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
   let mut handles = vec![];
   let chunk_size = 2;
   for chunk in numbers.chunks(chunk_size) {
      let handle = thread::spawn(|| {
         let sum: i32 = chunk.iter().sum();
         sum
```

```
    });

    handles.push(handle);

  }

  let mut total = 0;

  for handle in handles {

    let partial_sum = handle.join().unwrap();

    total += partial_sum;

  }

  println!("Total: {}", total);

}
```

Dica: Quando uma *thread* é criada com uma *closure,* como argumento, ela é passada por referência. O tipo *chunks* é do tipo &[i32] que é "emprestado" e está relacionado com a variável *numbers*. O resultado correto é 55.

### 6.2.1 Resolução B

```
use std::thread;

fn main() {

  let numbers = vec![1, 2, 3, 4, 5, 6, 7, 8, 9, 10];

  let mut handles = vec![];

  let chunk_size = 2;

  for chunk in numbers.chunks(chunk_size) {

    // to_vec() clona o chunk para um vetor

    let chunk = chunk.to_vec();

    let handle = thread::spawn(move || {

      let sum: i32 = chunk.iter().sum();

      sum

    });

    handles.push(handle);

  }
```

```
    let mut total = 0;

    for handle in handles {

        let partial_sum = handle.join().unwrap();

        total += partial_sum;

    }

    println!("Total: {}", total);

}
```

Neste código a *closure* não é passada por referência, com o uso do *move* e por isso as *threads* tornam se donas dos dados que recebem e isso resolve erros que o compilador apresenta para evitar referências que apontam para variáveis já destruídas. Outra mudança foi declarar um *chunk* com o método *.to_vec()* que transforma um *chunk* que é um valor emprestado num vetor que tem que ter dono. Estes problemas ocorrem porque *Rust* não permite passar referências de variáveis entre *threads*.

## 7. Desafio

Este desafio implica pesquisa de tópicos não presentes neste guião para incentivar trabalho autónomo. O objetivo é criar um programa que vigia um diretório (definido pelo programador) e organiza todos os ficheiros lá presentes por extensão, por exemplo, se criarmos um ficheiro *.txt* um diretório com o nome *TXT* é criado. Um requisito é pesquisar e utilizar a *crate* *notify*, também é necessário a aprendizagem autónoma sobre a utilização de ficheiros e criação de diretórios através da crate *fs*.

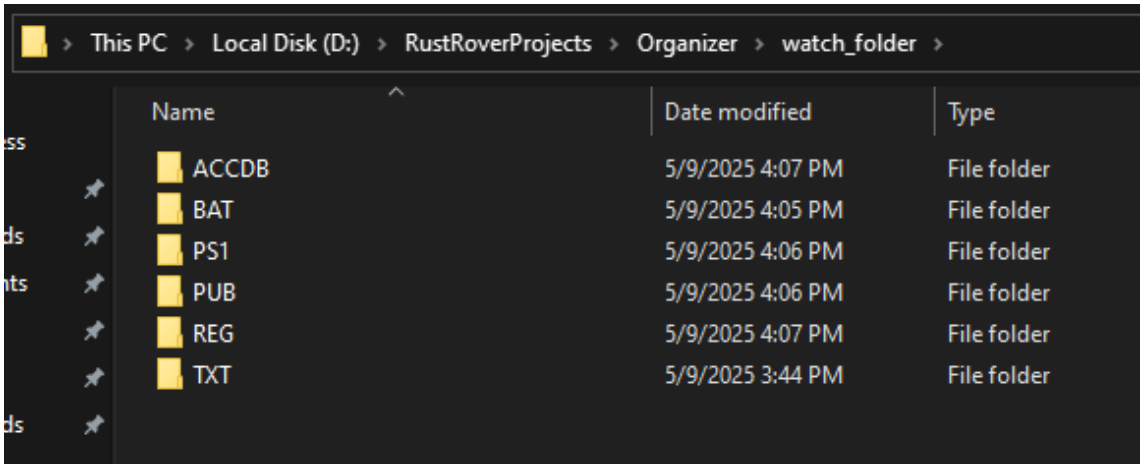



**Figura 26 - Exemplo do funcionamento**

## 8. Referências


[1] "Rust programming language," Rust Foundation, [Online]. Available: https://www.rust-lang.org/. [Acedido em 10 Abril 2025].

[2] "The rustc book," Rust Foundation, [Online]. Available: https://doc.rust-lang.org/rustc/. [Acedido em 10 Abril 2025].

[3] "Packages and Crates," The Rust Programming Language, [Online]. Available: https://doc.rust-lang.org/book/ch07-01-packages-and-crates.html. [Acedido em 10 Abril 2025].

[4] "The Cargo Book," Rust Foundation, [Online]. Available: https://doc.rust-lang.org/cargo/. [Acedido em 10 Abril 2025].

[5] "Rust as an ahead-of-time compiled language," University of Applied Sciences in Darmstadt, [Online]. Available: https://pascalbormann.de/asp_book/chap2/2_1.html. [Acedido em 10 Abril 2025].

[6] "What Is Ownership?," The Rust Programming Language, [Online]. Available: https://doc.rust-lang.org/book/ch04-01-what-is-ownership.html. [Acedido em 10 Abril 2025].

[7] "Rust By Example," Rust Foundation, [Online]. Available: https://doc.rust-lang.org/rust-by-example/scope/borrow.html. [Acedido em 10 Abril 2025].

[8] "Rust Compiler Development Guide," Rust Foundation, [Online]. Available: https://rustc-dev-guide.rust-lang.org/borrow_check.html. [Acedido em 10 Abril 2025].

[9] "The Rust Programming Language," Rust Foundation, [Online]. Available: https://doc.rust-lang.org/book/ch10-03-lifetime-syntax.html. [Acedido em 10 Abril 2025].

[10] "Module thread," Rust Foundation, [Online]. Available: https://doc.rust-lang.org/std/thread/index.html. [Acedido em 11 Abril 2025].

[11] K. Berryman, "Rust Programming Language Revolutionizes Embedded," Geisel Software, Inc, [Online]. Available: https://geisel.software/rust-revolutionizes-embedded/. [Acedido em 11 Abril 2025].

[12] "WebAssembly," Mozilla Foundation, [Online]. Available: https://developer.mozilla.org/en-US/docs/WebAssembly. [Acedido em 11 Abril 2025].

[13] Dinesh, "https://medium.com/@humble_bee/why-is-memory-safety-without-gc-a-big-deal-in-rust-41f6bdd5902f," Medium, [Online]. Available: https://medium.com/@humble_bee/why-is-memory-safety-without-gc-a-big-deal-in-rust-41f6bdd5902f. [Acedido em 11 Abril 2025].

[14] "Fearless Concurrency," The Rust Programming Language, [Online]. Available: https://doc.rust-lang.org/book/ch16-00-concurrency.html. [Acedido em 11 Abril 2025].

[15] "Fearless Concurrency with Rust," Rust Foundation, [Online]. Available: https://blog.rust-lang.org/2015/04/10/Fearless-Concurrency/. [Acedido em 11 Abril 2025].

[16] "Rust vs. C++: Differences and use cases explained," TechTarget, [Online]. Available: https://www.techtarget.com/searchapparchitecture/tip/Rust-vs-C-Differences-and-use-cases. [Acedido em 12 Abril 2025].

[17] J. Arundel, "Rust vs Go in 2025," Bitfield Consulting, [Online]. Available: https://bitfieldconsulting.com/posts/rust-vs-go. [Acedido em 12 Abril 2025].

[18] Rhymu, "Swift vs. Rust -- an Overview of Swift from a Rusty Perspective," [Online]. Available: https://dev.to/rhymu/swift-vs-rust-an-overview-of-swift-from-a-rusty-perspective-18c7. [Acedido em 12 Abril 2025].

[19] "Install Rust," The Rust Programming Language, [Online]. Available: https://www.rust-lang.org/tools/install. [Acedido em 10 Abril 2025].

[20] "Variables and Mutability," The Rust Programming Language, [Online]. Available: https://doc.rust-lang.org/book/ch03-01-variables-and-mutability.html. [Acedido em 10 Abril 2025].

[21] "Data Types," The Rust Programming Language, [Online]. Available: https://doc.rust-lang.org/book/ch03-02-data-types.html. [Acedido em 10 Abril 2025].

[22] "Functions," The Rust Programming Language, [Online]. Available: https://doc.rust-lang.org/book/ch03-03-how-functions-work.html. [Acedido em 15 Abril 2025].

[23] "Control Flow," The Rust Programming Language, [Online]. Available: https://doc.rust-lang.org/book/ch03-05-control-flow.html. [Acedido em 10 Abril 2025].

[24] "References and Borrowing," The Rust Programming Language, [Online]. Available: https://doc.rust-lang.org/book/ch04-02-references-and-borrowing.html. [Acedido em 16 Abril 2025].

[25] "Defining and Instantiating Structs," The Rust Programming Language, [Online]. Available: https://doc.rust-lang.org/book/ch05-01-defining-structs.html. [Acedido em 15 Abril 2025].

[26] "Using Threads to Run Code Simultaneously," The Rust Programming Language, [Online]. Available: https://doc.rust-lang.org/book/ch16-01-threads.html. [Acedido em 29 Abril 2025].

[27] "Shared-State Concurrency," The Rust Programming Language, [Online]. Available: https://doc.rust-lang.org/book/ch16-03-shared-state.html. [Acedido em 29 Abril 2025].

[28] "Introduction," Rust By Example, [Online]. Available: https://www.rust-lang.org/tools/install. [Acedido em 10 Abril 2025].